

Swift MM Quick Integration Guide

[Overview](#)

[Swift Order Subscriber](#)

[Initializing and Subscribing](#)

[\(Recommended\) Integrating with JIT](#)

[Manually Constructing Transactions without JIT](#)

[PrecedingIxs/CustomIxIndex](#)

[Deriving Function Arguments From Swift Messages](#)

[Publishing Indicative Quotes](#)

Overview

Drift Swift offers a new way to receive and trade against taker orders.

Prior to Swift, market makers were notified of new taker orders via solana RPC updates, as takers needed to first land their order transactions onchain. With Swift, MMs can receive signed market order messages from takers directly via a Swift websocket server. Makers then include the message within their transactions in order to place them on behalf of the taker and trade against them ('place and make').

Note: Swift is only available for market orders for perps initially. Expansion coming soon.

Swift Order Subscriber

Initializing and Subscribing

The most straightforward way to connect to the websocket server and begin receiving messages is via the `SwiftOrderSubscriber` which is available within the Python, Rust, and Typescript drift sdks (along with all necessary arguments and types). An example of constructing the `SwiftOrderSubscriber` is below:



```
const driftClient = new DriftClient(driftClientConfig); const userMap = new
UserMap({ driftClient, connection, subscriptionConfig: { type: 'websocket',
resubTimeoutMs: 30_000, commitment: 'confirmed' }, skipInitialLoad: false,
includeIdle: false, }); await driftClient.subscribe() await
userMap.subscribe() const swiftOrderSubscriber = new SwiftOrderSubscriber({
driftEnv: 'mainnet-beta', marketIndexes: [0, 1, 2, 3], /** In the future,
this will be used for verifying $DRIFT stake as we add authentication for
delegate signers For now, pass a new keypair or a keypair to an empty wallet
*/ keypair: new Keypair(), driftClient, userAccountGetter: userMap, });
```

To subscribe and begin using messages, call `subscribe()`, and provide it an `onOrder()` callback that will be invoked on every new order.

The `onOrder` callback has 2 arguments: the raw message received from the websocket server (the raw message is necessary to supply as ix data to the drift program), and the deserialized order data for inspection: [SignedMsgOrderParamsMessage](#).

This subscription is taken care of under the hood when integrating with the `JitProxyClient` (detailed in the next section). A simple callback example is provided below.



```

await swiftOrderSubscriber.subscribe( async ( orderMessageRaw: any,
signedMessage: | SignedMsgOrderParamsMessage |
SignedMsgOrderParamsDelegateMessage, isDelegateSigner?: boolean ) => { const
takerAuthority = new PublicKey(orderMessageRaw['taker_authority']); const
takerUserPubkey = isDelegateSigner ? (signedMessage as
SignedMsgOrderParamsDelegateMessage).takerPubkey : await
getUserAccountPublicKey( this.driftClient.program.programId, takerAuthority,
(signedMessage as SignedMsgOrderParamsMessage).subAccountId ); const
orderParams = signedMessage.signedMsgOrderParams; console.log(
`${takerUserPubkey.toString()} wants to ${getVariant( orderParams.direction
)} ${convertToNumber( orderParams.baseAssetAmount, BASE_PRECISION )} units of
market index ${orderParams.marketIndex}` ); console.log( `Auction params: ${
orderParams.auctionDuration } slots. Start Price: ${convertToNumber(
orderParams.auctionStartPrice!, PRICE_PRECISION )}. End Price:
${convertToNumber( orderParams.auctionEndPrice!, PRICE_PRECISION )}` ); } );

```

(Recommended) Integrating with JIT

Swift is essentially an off-chain JIT extension. When takers sign a message, they include the slot when the order's auction begins. This slot is used as the auction start slot and for calculating the order's current price when the order is posted on-chain. The orders are posted either as part of a fill or when a maker trades against it.

The simplest way to abstract over the on-chain and off-chain order flows, is with the `JitProxyClient`. It uses the jit-proxy program (a wrapper program for drift) that makes it easier to market make against auction orders. To learn more about JIT or JitProxy, see the following links:

- [Drift matching engine](#)
- [JIT Maker FAQ](#)
- [Jit Proxy SDK](#)

For any additional questions on JIT, please ping the team on telegram or discord.

To integrate Swift with JIT, simply pass a `SlotSubscriber` and `SwiftOrderSubscriber` to either the `JitterSniper` or `JitterShotgun`.



One important thing to note when integrating Swift with Jitters is that orders will also come through the `AuctionSubscriber` after the order lands on-chain. For example, consider the following scenario:

1. Swift order gets sent to market maker for 100 SOL-PERP
2. Swift order arrives to your bot via `SwiftOrderSubscriber`
3. One market maker fills the size for 1 SOL-PERP and in the process posts the order on-chain
4. Order comes through the `AuctionSubscriber` with base amount filled size 1 and base amount remaining as size 99

In the above example, you can get “notified” or have callbacks executed twice for the same Swift Order — once while it’s off-chain and again when it’s on-chain. The drift sdk provides a method called `isSignedMsgOrder` that will detect if orders coming through the `AuctionSubscriber` were Swift orders, and you can then filter your callbacks as necessary. For convenience, the jitters accept a boolean config `auctionSubscriberIgnoresSwiftOrders`, that causes the jitters to ignore orders coming through the `AuctionSubscriber` where `isSignedMsgOrder` evaluates to true. This is set to true in the example below

```
const swiftOrderSubscriber = new SwiftOrderSubscriber({ driftEnv: 'mainnet-beta', marketIndexes: [0, 1, 2, 3], keypair: myKeypair, driftClient,
userAccountGetter: userMap, }); const slotSubscriber = new
SlotSubscriber(connection, { resubTimeoutMs: 30_000 }); const jitProxyClient
= new JitProxyClient({ driftClient, programId: new
PublicKey(sdkConfig.JIT_PROXY_PROGRAM_ID!), }); const jitter = new
JitterSniper({ auctionSubscriber, driftClient, jitProxyClient,
swiftOrderSubscriber, slotSubscriber, auctionSubscriberIgnoresSwiftOrders:
true, }); await jitter.subscribe();
```

The jitter will handle all the message parsing, transaction construction, and transaction sending when integrating with a JitMaker strategy. For an example of using the Jitters within a JitMaker bot, see the example open sourced bot [here](https://github.com/drift-protocol/jit-maker-bot).



Manually Constructing Transactions without JIT

For Swift to be secure, the Drift program uses transaction introspection. This process checks an 'Ed25519 Verify Ix' is present and succeeds in verifying the taker's order message against the taker's signature. The Ed25519 Verify instruction data is essentially offset pointers into instruction data from a different custom instruction - this tells the verify instruction where to look in the ix data for the taker's wallet public key, the message signed by the taker, and the instruction itself. This can be quite tricky which is why its recommend to use the jitters to abstract this process.

The result of the above is that in order to place the taker's order on-chain and fill/make against it atomically, use the DriftClient to construct two instructions: an `ed25519` instruction, and a `place_signed_msg_taker_order` instruction.

Importantly, these must be ordered prior to any fill or place and make instruction.

When looking to market make against a Swift order, its is recommend to use the DriftClient method `getPlaceAndMakeSignedMsgPerpOrderIxs` — It will construct all ixs: the verify ix, the place taker order ix, and place and make instruction. Here are the function arguments:

```
public async getPlaceAndMakeSignedMsgPerpOrderIxs(
    signedSignedMsgOrderParams: SignedMsgOrderParams, signedMsgOrderUuid:
    Uint8Array, takerInfo: { taker: PublicKey; takerStats: PublicKey;
    takerUserAccount: UserAccount; signingAuthority: PublicKey; }, orderParams:
    OptionalOrderParams, referrerInfo?: ReferrerInfo, subAccountId?: number,
    precedingIxs: TransactionInstruction[] = [], overrideCustomIxIndex?: number )
```

PrecedingIxs/CustomIxIndex



The example below this section shows how to extract most of the information necessary for the `getPlaceAndMakeSignedMsgPerpOrderIx` function. But before this, an important thing to note is the `precedingIx` and `overrideCustomIxIndex` arguments in this function. It is very important to set these correctly as they tell the verify ix which instruction to check for the message data. Incorrectly setting these values will cause the transaction to fail. It is recommend to build instructions in a "sequential" order, and pass the `precedingIx` argument — that way it is known at instruction creation time where the verify + custom ix are in the resulting transaction. pseudo code example:

```
const ix = []; ix.push(ComputeBudgetIx: set limit)
ix.push(ComputeBudgetIx: set price (or jito tip or both))
const placeAndMakeIx = getPlaceAndMakeSignedMsgPerpOrderIx(..., ix);
ix.push(placeAndMakeIx)
```

In case you do not create instructions this way, we also allow setting the `overrideCustomIxIndex`, which allows you to pass an empty array into the `precedingIx` and then set the index of the place signed taker order ix manually. So in the above example, you would use `getPlaceAndMakeSignedMsgPerpOrderIx` with a value of 3 — in the following array, the custom ix is at the 3 index [compute budget, compute budget, verify, place signed taker order, place and make].

Deriving Function Arguments From Swift Messages

Below is an example of deriving some bare bones information from within a callback of the Swift order subscriber. It naively generates ix for trading against the order:



```

await swiftOrderSubscriber.subscribe( async ( orderMessageRaw: any,
signedMessage: SignedMsgOrderParamsMessage, isDelegateSigner?: boolean, ) =>
{ const signedMsgOrderParamsBufHex = Buffer.from(
orderMessageRaw['order_message'] ); const signedMsgOrderParams =
signedMessage.signedMsgOrderParams; const takerAuthority = new PublicKey(
orderMessageRaw['taker_authority'] ); const signingAuthority = new PublicKey(
orderMessageRaw['signing_authority'] ); const takerUserPubkey =
isDelegateSigner ? (signedMessage as
SignedMsgOrderParamsDelegateMessage).takerPubkey : await
getUserAccountPublicKey( this.driftClient.program.programId, takerAuthority,
(signedMessage as SignedMsgOrderParamsMessage).subAccountId ); const
takerUserPubkeyString = takerUserPubkey.toBase58(); const takerUserAccount =
( await this.userMap.mustGet( takerUserPubkey.toString() )
).getUserAccount(); const isOrderLong =
isVariant(signedMsgOrderParams.direction, 'long'); const ixes = await
this.driftClient.getPlaceAndMakeSignedMsgPerpOrderIx( { orderParams:
signedMsgOrderParamsBufHex, signature:
Buffer.from(orderMessageRaw['order_signature'], 'base64'), },
decodeUTF8(orderMessageRaw['uuid']), { taker: takerUserPubkey,
takerUserAccount, takerStats: getUserStatsAccountPublicKey(
this.driftClient.program.programId, takerUserAccount.authority ),
signingAuthority, }, getLimitOrderParams({ marketType: MarketType.PERP,
marketIndex: signedMsgOrderParams.marketIndex, direction: isOrderLong ?
PositionDirection.SHORT : PositionDirection.LONG, baseAssetAmount:
signedMsgOrderParams.baseAssetAmount, price: isOrderLong ?
signedMsgOrderParams.auctionStartPrice!.muln(99).divn(100) :
signedMsgOrderParams.auctionEndPrice!.muln(101).divn(100), postOnly:
PostOnlyParams.MUST_POST_ONLY, immediateOrCancel: true, }) ); } )

```

An example of the above process can be seen in our Swift maker example bot which you can find here:



src/experimental-bots/swift/makerExample.ts drift-labs/keeper-bots-v2
master



Publishing Indicative Quotes